

DE JSON A APIS

Networking y Webservices

Fundamentos de redes y consumo de la API DCCSteam.



¿Qué es JSON?



JavaScript Object Notation

Un formato de texto ligero y universal para **enviar información de un computador a otro**. Es independiente del lenguaje de programación.

Métodos de Serialización

`json.dumps()`

Convierte un objeto de Python (Ej: Diccionario) a un **String** en formato JSON. (Ideal para enviar por red).

`json.loads()`

Convierte un **String** JSON recibido a un objeto de Python (Diccionario/Lista). (Ideal al recibir de la red).

`json.dump()`

Convierte un objeto de Python y lo guarda directamente en un **Archivo físico** (.json).

`json.load()`

Lee un **Archivo físico** (.json) y lo convierte en un objeto de Python.

Tipos de Datos Serializables

Tipo en Python	Transformación en JSON	Ejemplo Resultante
dict	Objeto (Object)	{"clave": "valor"}
list, tuple	Arreglo (Array)	[1, 2, 3]
str	Cadena (String)	"Texto"
int, float	Número (Number)	42 o 3.14
True, False	Booleano (Boolean)	true, false
None	Nulo (Null)	null

¿Cómo se comunican los computadores?

Necesitamos 5 elementos clave:

1. Un **medio** para transmitir señales, como Ethernet, fibra óptica, Wi-Fi o redes móviles.
2. Una forma de identificar la interfaz de destino, como una **dirección IP**.
3. Una forma de identificar el proceso de destino: un **puerto**.
4. Un **protocolo de transporte**, como TCP o UDP.
5. Un **protocolo de aplicación**, que define el significado y formato de los mensajes, como HTTP, DNS o SMTP.

Dirección IP (Internet Protocol)



Es un identificador único asignado a cada dispositivo conectado a una red.

Características: Es una dirección lógica, puede ser dinámica o estática, y permite el enrutamiento de paquetes de datos desde el origen al destino correcto.

Tipos de Direcciones IP

IPv4

- **Tamaño:** 32 bits.
- **Combinaciones:** ~4.3 mil millones de direcciones. (Agotándose).
- **Formato:** 4 bloques numéricos separados por puntos.

Ejemplo:

192.168.1.1

IPv6

- **Tamaño:** 128 bits.
- **Combinaciones:** ~340 undecillones (3.4×10^{38}).
- **Formato:** 8 bloques hexadecimales separados por dos puntos.

Ejemplo:

2001:0db8:85a3:0000:0000:8a2e:0370:7334

DNS (Domain Name System)



El directorio telefónico de Internet. **Traduce** nombres de dominio legibles por humanos a las direcciones IP requeridas por las máquinas.

www.google.com ⇔ **142.250.190.46**

Puertos

Mientras la IP identifica la casa (el computador), el **Puerto** identifica la puerta de la habitación específica (el proceso/aplicación). Permite múltiples servicios simultáneos en una misma máquina.

Puerto	Servicio Común
80	HTTP (Web no segura)
443	HTTPS (Web segura)
22	SSH (Conexión remota)
5000 / 5001	Flask (Desarrollo Python)

Protocolos de Transporte: TCP vs UDP



TCP (Transmission Control Protocol)

Orientado a conexión y confiable.

- Verifica que los paquetes lleguen.
- Mantiene el orden original.
- Pide retransmisión si hay pérdida.
- **Usos:** Web (HTTP), Email, Archivos.



UDP (User Datagram Protocol)

Sin conexión y ultra rápido.

- No garantiza entrega (puede haber pérdida).
- No garantiza el orden.
- Sin latencia de confirmación.
- **Usos:** Streaming, Videollamadas, Gaming.

Tabla de Conceptos de Redes

Elemento	¿Qué es?	Analogía
Medio	Canal físico o inalámbrico de transmisión.	La carretera.
IP	Identificador único de la máquina (v4 o v6).	La dirección de la casa.
Puerto	Identificador del proceso o servicio.	El número de departamento.
DNS	Traductor de Nombres a IPs.	El directorio telefónico.
TCP/UDP	Protocolos de cómo viaja la data.	Correo certificado vs Postal rápida.

El Modelo Cliente-Servidor



El Cliente

Aplicación o usuario que **solicita** información o acciones. (Ej: Nuestro script de Jupyter Notebook, un navegador).



El Servidor

Máquina que **escucha** peticiones, procesa datos y devuelve respuestas. (Ej: Nuestro archivo `server.py` en ejecución).

¿Qué es una API?

Application Programming Interface

Un conjunto de reglas y contratos que permite que dos aplicaciones de software se comuniquen entre sí. La API actúa como un **puente** estandarizado, ocultando la complejidad interna del servidor.

La Analogía del Restaurante



El Cliente

(La Mesa/Comensal)

Hace el pedido de lo que necesita.



La API

(El Mesero)

Toma la orden, la lleva atrás y trae la comida.



El Servidor

(El Cocinero/Cocina)

Prepara la comida y accede a los ingredientes (BD).

Estructura de una URL

```
https://api.dccsteam.cl:5001/juegos/busqueda?  
genero=accion
```

Esquema: Protocolo usado (http o https).

Host: Dominio o IP del servidor.

Puerto: Puerta de entrada (Opcional, asume 80/443).

Ruta (Path): Recurso específico a acceder.

Query String: Parámetros extra separados por &. Inicia con ?.

Métodos HTTP (1/2)

GET

Propósito: Leer o recuperar datos.

- No modifica el estado del servidor.
- Los parámetros viajan en la URL (Query String).
- No lleva cuerpo (Body).

POST

Propósito: Crear nuevos recursos.

- Añade nueva información.
- Los datos complejos viajan en el Body (JSON).
- No es idempotente (repetirlo crea duplicados).

Métodos HTTP (2/2)

PUT

Propósito: Reemplazo total.

- Sobrescribe el recurso completo.

PATCH

Propósito: Modificación parcial.

- Solo actualiza campos específicos (Ej: cambiar rating).

DELETE

Propósito: Eliminar recurso.

- Borra la información del servidor. Requiere alta seguridad.

Códigos de Estado (Status Codes)

1xx (Informacional): Petición recibida, continuando proceso.

2xx (Éxito): Petición recibida, entendida y aceptada.
(Ej: 200 OK, 201 Created)

3xx (Redirección): Se requiere acción adicional.

4xx (Error Cliente): Sintaxis incorrecta o no autorizado.
(Ej: 400 Bad Request, 404 Not Found)

5xx (Error Servidor): El servidor falló al cumplir una petición válida. (Ej: 500 Internal Error)

Estructura de una Solicitud (HTTP Request)

Partes enviadas por el Cliente:

- 1. URL / Endpoint:** La dirección a la que vamos (Ej: /juegos).
- 2. Método (Verbo):** La acción deseada (GET, POST, etc.).
- 3. Headers (Cabeceras):** Metadatos clave-valor. Proveen contexto adicional, como el formato o Tokens de Autorización.
- 4. Body (Cuerpo):** Datos enviados al servidor (Ej: JSON con datos de un nuevo juego).

**Solo en métodos como POST/PUT/PATCH.*

Estructura de una Respuesta (HTTP Response)

Partes devueltas por el Servidor:

- 1. Status Code:** El código numérico (200, 404) que indica el resultado de la petición.
- 2. Headers (Cabeceras):** Metadatos del servidor. Indican, por ejemplo, el Content-Type (si el body es JSON o HTML).
- 3. Body (Cuerpo):** La información solicitada (lista de juegos) o el mensaje de error explícito en caso de fallos.

Preparando una Request

 **URL Base:** Dirección principal del servidor web.

 **Endpoints:** Rutas específicas hacia un recurso.

 **Contrato:** Define qué datos enviar y recibir.

`https://api.host.com/v1/usuarios`

URL Base

Endpoint

Petición GET

Propósito: Recuperar información del servidor.

Módulo: Método `requests.get()`.

Params: Diccionario convertido a Query String.

```
import requests

# Endpoint genérico
url = "https://host.com/juegos"
parametros = {"genero": "accion"}

# Ejecución de la petición
respuesta = requests.get(
    url,
    params=parametros
)
```

Atributos de la Respuesta

.status_code

Código numérico de estado.
Ejemplo: 200 (OK).

.json()

Convierte cuerpo a
diccionario.
Extrae datos fácilmente.

.headers

Metadatos del servidor.
Incluye Content-Type.

Petición POST

Propósito: Enviar datos para crear recursos.

El Body: Carga útil de datos principal.

Argumento JSON: Serializa el diccionario automáticamente.

```
import requests

url = "https://host.com/juegos"

# El Body como diccionario
nuevo_juego = {
    "nombre": "Portal 3",
    "precio": 29.99
}

# Ejecución de la petición
respuesta = requests.post(
    url,
    json=nuevo_juego
)
```

Autenticación con Tokens



Acceso Restringido

Las APIs seguras requieren credenciales enviadas en los Headers.

```
# Definimos credenciales
credenciales = {
    'Authorization': 'IIC2233-Token'
}

# Enviamos en la petición
resp = requests.get(
    url,
    headers=credenciales
)
```

Instanciación de Flask

```
# Importamos el módulo
from flask import Flask

# Creamos la aplicación
app = Flask(__name__)
```

Flask: Framework minimalista para crear servidores web.

__name__: Identifica el módulo actual para ubicar recursos.

Decorando Endpoints

Método Único (GET)

```
# Por defecto es GET
@app.route('/juegos')
def obtener_juegos():
    return "Lista"
```

Múltiples Métodos

```
# Responde a GET y POST
@app.route('/juegos',
          methods=['GET',
                  'POST'])
def gestionar_juegos():
    pass
```

Rutas Dinámicas

```
# Recibe un id entero
@app.route('/juegos/')
def detalle_juego(id):
    return f"ID: {id}"
```

Métodos Útiles en Flask

`request.get_json()`

Lee body entrante y convierte a diccionario.

```
body = request.get_json()
```

`request.args.get()`

Extrae parámetros de la Query String.

```
filtro = request.args.get('genero')
```

`abort()`

Detiene ejecución y lanza código de error.

```
if not user: abort(404)
```

Instanciando una Response

Todo endpoint debe retornar un objeto **Response()**.

response: El cuerpo, enviado como String JSON.

status: Código numérico de estado HTTP.

content_type: Formato de datos enviado.

Valores comunes de content_type:

'application/json' - Data JSON
(Dicts/Listas)

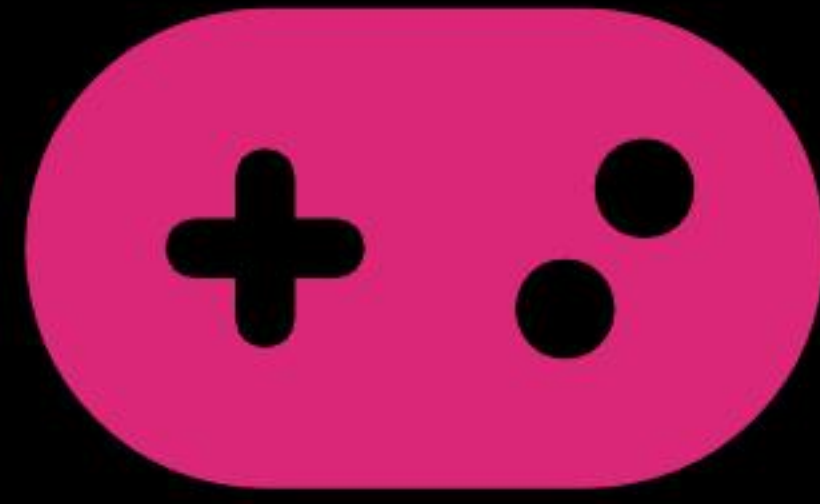
'text/html' - Páginas Web en HTML

'text/plain' - Texto simple sin formato

```
from flask import Response
import json

# Datos de ejemplo
datos = {"mensaje": "Éxito"}

# Retorno al cliente
return Response(
    response=json.dumps(datos),
    status=200,
    content_type='application/json'
)
```



DCCSteam API Lista

Conoces las bases de la red, cómo hablar en HTTP, y el código para enlazar Flask y Requests.